Lesson 2 Teacher's Key

TIC-80

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Full cart:

```
x=60 y=60
function TIC()
 cls(0)
 if btn(3) then x=x+1 end
 spr(1,x,y,0)
end
```

Level 2 - Complete Worked Solution

All 4 directions using the standard button indices (0=Up, 1=Down, 2=Left, 3=Right):

```
x=60 y=60
function TIC()
 cls(0)
 if btn(0) then y=y-1 end
 if btn(1) then y=y+1 end
 if btn(2) then x=x-1 end
 if btn(3) then x=x+1 end
 spr(1,x,y,0)
end
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking (or if a student blows past all of them, this is your own jumping-off point for exploring further right alongside them).

Animate: blink while moving, static when idle

This solution uses every parameter of spr(): spr(id, x, y, colorkey, scale, flip, rotate, w, h) - confirmed against TIC-80's own spr() documentation. The two new ones are w and h: how many 8x8 tiles wide/tall to draw as one connected image (both default to 1). Setting w=2, h=2 draws a 2x2 block of tiles (16x16 pixels) starting at index 1 - a "composite sprite" - instead of the single top-left 8x8 tile that spr(1,x,y,0) was drawing before. In the Sprite Editor (F2, or the monster-icon button), draw your character across a full 2x2 block of tiles rather than a single tile, so the four cells read as one connected image instead of a quarter of one.

Per TIC-80's spr() documentation: "Fractional factors are floored to integers, and both a factor of 0 and negative factors cause the sprite to not get drawn." That's the trick behind the blink -

alternating the scale argument between 0 and 1 makes the sprite blink on and off, no second image needed.

f is a flash-toggle counter that climbs by 0.1 every frame; math.floor(f)%2 flips between 0 and 1 roughly 3 times a second (at 60fps). While a direction is held, s takes that flipping value (blink); when idle, s is pinned to 1 (always visible):

```
f = 0  -- flash toggle
...
f=f+0.1  -- advance flash speed every frame
if btn(3) then
  s=math.floor(f)%2  -- blinks 0/1 while moving
else
  s=1  -- always visible when idle
end
...
spr( 1, x, y, 0, s, 0, 0, 2, 2 )  -- s (scale) is what makes it blink
```

Sound: beep-beep while moving, silent when idle

The goal here is a repeating beep-beep for as long as a direction is held, stopping the instant it's released - a different pattern than a single blip on press.

b is a beep-toggle counter climbing by 0.2 every frame; math.floor(b)%2 alternates between sound slots 0 and 1 roughly every 5 frames, so you need two short SFX slots set up (e.g. two quick blips at slightly different pitches) for the alternation to read as a beep-beep rather than one repeated note.

This calls sfx() every single frame a direction is held (not just on the frame it's first pressed) - that's intentional here, not a bug, because the goal is a continuous sound. Since a plain sfx() call has no built-in "note off" (duration defaults to play-forever), the idle/else branch explicitly calls sfx(-1) to silence channel 0 the moment no direction key is down:

```
b = 1  -- beep toggle
...
b=b+0.2  -- advance beep speed every frame
if btn(3) then sfx(math.floor(b)%2)  -- alternates sound 0/1 for a beep-
beep while held
else sfx(-1) end  -- stop channel 0 the instant no direction key is down
```

Recolor: different color per direction, reset when idle

Worth calling out to the class: every other engine in this lesson does Recolor by swapping in a whole separate pre-made colored image, because their color-tint functions (image_blend, modulate, spriteRenderer.color, and so on) only give a clean result on white source art, and muddy on anything else - swapping images sidesteps that entirely. TIC-80 doesn't have that problem, so it doesn't need that workaround: poke4() rewrites an actual palette slot, not a multiplied tint, so the recolor is exact regardless of what the sprite's original color was. That makes TIC-80 the one engine in the course where true live recoloring is both possible and genuinely simpler than swapping images - a real difference in what the hardware/engine offers, not an arbitrary inconsistency.

Heads up: TIC-80 has no pal()/palette()/palset() function - that's a PICO-8 thing. TIC-80's own documentation for spr() has a section called "Color Swapping Sprites" that uses exactly this poke4() + PALETTE_MAP technique, so this is the officially intended way to do a live recolor.

In the Sprite Editor (F2, or the monster-icon button), the color palette is the strip in the lower-left. Click through it to see which index is which color, and note which index makes up most of your character's fill - in this cart that's index 10.

PALETTE_MAP (0x3FF0) is a magic-looking value, but it has a plain explanation: it's a byte address in VRAM. poke4() writes in 4-bit nibbles, and TIC-80's own peek/poke documentation explains that reaching a byte address in nibble units means multiplying by 2 (each byte is 2 nibbles). So PALETTE_MAP*2 gets you to the start of that byte's nibbles, and adding the original color index (0-15) selects which of the 16 palette slots you're overriding - poke4(PALETTE_MAP*2+10, 6) means "wherever color 10 would be drawn, draw color 6 instead."

Important: this doesn't touch the actual palette entries - it's a live remap applied while drawing. Color 10 still exists; you're just redirecting it. That's why swapping index 10 to 6 and later poking index 10 back to 10 (itself) fully restores the original look - nothing was destroyed, just temporarily overridden:

```
PALETTE_MAP=0x3FF0
...
if btn(3) then poke4(PALETTE_MAP*2+10,6)  -- while held, redirect color 10
to green (6)
else poke4(PALETTE_MAP*2+10,10) end  -- idle: redirect color 10 back to
itself - fully restored
```

Clamp movement to the screen

TIC-80's screen is 240x136; clamping to 0-232/0-128 keeps the sprite fully on screen. Placed ahead of the if/elseif/else block so it applies every frame regardless of which branch runs:

```
 x=math.max(0,math.min(232,x))
 y=math.max(0,math.min(128,y))
```

Structure: group everything per trigger in one place

The most important structural idea in this version: every goal above is folded into the SAME single if/elseif/else block that already handles movement from Level 2, rather than writing separate blocks that each re-check btn() on their own. Each direction's branch now does its movement, its recolor, its beep, and its blink state together in one place - and the else (idle) branch resets color, sound, and scale together too. That keeps "what happens when I press Right" fully readable in one spot instead of scattered across separate checks:

```
if btn(3) then -- right: move + recolor + beep + blink, all together
  x=x+spd
  poke4(PALETTE_MAP*2+10,6)
  sfx(math.floor(b)%2)
  s=math.floor(f)%2
end
```

Complete Level 3 solution (all 5 goals combined)

The full working cart, combining every goal above - movement, blink, recolor, and beep-while-held - inside one if/elseif/else block, plus the clamp-to-screen check. The indentation and comment alignment are preserved exactly as authored, since the staggered spacing is what makes the branches easy to scan at a glance:

```
-- title:   Mini-Game
-- author:  Robert Kuropkat
-- desc:    Semester 01 project
```

```
-- site:    https://doersofstuff.com/
-- license: MIT License (change this to your license of choice)
-- version: 0.1
-- script:  lua

PALETTE_MAP=0x3FF0

x   = 60  -- starting x-position
y   = 60  -- starting y-position
spd = 1   -- speed
b   = 1   -- beep toggle
f   = 0   -- flash toggle
s   = 1   -- sprite scale

function TIC()

   cls(0)

 b=b+0.2  -- set beep speed
 f=f+0.1  -- set flash speed

 x=math.max(0,math.min(232,x))
 y=math.max(0,math.min(128,y))

 if     btn(0) then -- up
               poke4(PALETTE_MAP*2+10,3)
               sfx(math.floor(b)%2)
               y=y-spd
               s=math.floor(f)%2
 elseif btn(1) then -- down
               poke4(PALETTE_MAP*2+10,1)
               sfx(math.floor(b)%2)
               y=y+spd
               s=math.floor(f)%2
 elseif btn(2) then -- left
               poke4(PALETTE_MAP*2+10,0)
               sfx(math.floor(b)%2)
               x=x-spd
               s=math.floor(f)%2
 elseif btn(3) then -- right
               poke4(PALETTE_MAP*2+10,6)
               sfx(math.floor(b)%2)
               x=x+spd
               s=math.floor(f)%2
 else          poke4(PALETTE_MAP*2+10,10)
               sfx(-1)
               s=1
 end

   spr( 1  -- index
      , x  -- x-position
                , y  -- y-position
                , 0  -- color key
                , s  -- scale
                , 0        -- flip
                , 0  -- rotate
```

```
                , 2  -- width in sprites
                , 2  -- height in sprites
                )

end
```